

ICPC Notebook Java

A progressive team reference for Java 17

A progressive algorithm reference for team practice

Profile: Beginner | 11 entries

Contents

How to use this reference **1**
Learning path . . . 1
Choosing an algorithm . . . 1
Java contest checklist . . . 1

Problem-solving foundations **2**

Guided problems **3**

1 Contest setup **4**
1.1 Standalone Main template . . . 4
1.2 Reusable fast input . . . 5

2 Fundamentals **5**
2.1 Loops, arrays, prefix sums, and brute force . . . 6
2.2 Maps, sets, and stacks . . . 7
2.3 Recursion and exhaustive subsets . . . 8
2.4 Binary search and bounds . . . 8
2.5 Interval scheduling and covering . . . 10

3 Dynamic programming **11**
3.1 First dynamic programs . . . 11

4 Graphs **12**
4.1 Adjacency lists, BFS, and grid flood fill . . . 12

5 Strings **13**
5.1 Characters, frequencies, and direct matching . . . 13

6 Number theory and combinatorics **14**
6.1 GCD, trial division, divisors, and powers . . . 14

How to use this reference

Read the algorithm's use case, cost, and caution before copying its code. The listing is the same Java source compiled by the repository tests. Use the exporter to combine the selected classes into one judge-ready Main.java.

Learning path: Beginner

Who this is for Teams learning to turn a problem statement into loops, data structures, and a correct first solution.
Before starting Java variables, methods, arrays, loops, conditionals, and basic input/output.
Practice method For each pattern, solve one small problem directly, test empty and boundary inputs, then explain the invariant in one sentence.

Learning goals

- Read constraints and estimate whether O(n), O(n log n), or O(n^2) fits.
- Implement and test simple loops, prefix sums, sorting, and direct search.
- Recognize a correct greedy choice and the four parts of a dynamic program.
- Represent a graph and use BFS for unweighted distances.

Conventions

- Vertices and arrays are zero-indexed.
- Ranges are half-open [left, right) unless documented otherwise.
- Check overflow, input bounds, and recursion depth before using a snippet.
- Choose the simplest algorithm that fits the problem constraints.

Choosing an algorithm

Problem signal	Choice	Typical cost
Small sequence, simple condition	Direct scan or brute force	O(n) or O(n^2)
Frequency or membership lookup	HashMap or HashSet	Expected O(1) per operation
All subsets of a small set	Backtracking	O(2^n)
Static interval sums	Prefix sums	O(n) build, O(1) query
Unweighted graph or grid	Breadth-first search	O(V+E)

Java contest checklist

- Check time and memory limits before choosing object-heavy structures.
- Use long for sums, distances, products, and counts that may exceed int.
- Check whether input is ASCII tokens or requires Unicode handling.
- Watch recursion depth on long paths; use an explicit stack if needed.
- Confirm whether edges are directed, whether weights can be negative, and whether ranges are inclusive.
- Test empty, singleton, duplicate, disconnected, and boundary cases.

Problem-solving foundations

Read constraints before coding

A solution is useful only when it fits the input size. One loop over n items is O(n); nested loops over every pair are O(n^2). Sorting is usually O(n log n). Exponential subset enumeration becomes expensive quickly.

Example At n = 2,000, trying roughly four million pairs may be reasonable; at n = 200,000, forty billion pairs are not. Check the actual time limit and Java overhead.
Try it For each problem, write n and a rough operation count next to the proposed approach.

Move from a baseline to a faster method

First state a simple correct method. Identify repeated work, then reuse it with a prefix array, sorted order, a map, or a maintained state. Keep the baseline as a test oracle for tiny cases.

Example Summing each query interval with a loop costs O(length). Building prefix sums once costs O(n), then each interval needs one subtraction.
Try it Implement both methods and compare every interval on a short random array.

State the invariant

An invariant describes what remains true after each loop step. It helps prove that a scan, binary search, queue, or greedy choice is correct.

Example During BFS, every queued vertex has a known shortest distance, and vertices leave the queue in nondecreasing distance.
Try it Explain why the first discovered path to a vertex has the fewest edges.

Check boundaries systematically

Test the smallest legal size, empty containers when permitted, duplicate values, negative values, and the largest numeric result. Keep zero-based indices and half-open ranges explicit.

Example For prefix sums of three values, prefix has four entries; the whole array is [0,3), and an empty range is [2,2).
Try it Write tests for an empty array, one value, and a range ending at n.

Guided problems

Answer many range sums

Task Given values [3, -2, 5, 4], answer sums for half-open ranges [1,3) and [0,4).
Result 3 and 10
Reasoning Build prefix [0,3,1,6,10]. A range [l,r) is prefix[r] - prefix[l], so the answers are 6 - 3 and 10 - 0.
Use BeginnerArrays.prefixSums and rangeSum
Cost O(n) build and O(1) per query.
Check Compare against a loop for all ranges, including [2,2) and [0,4).

Choose compatible activities

Task Given [0,3), [1,2), [2,4), and [4,5), choose the maximum number of non-overlapping intervals.
Result 3 activities
Reasoning Sort by ending time. Choose [1,2), then [2,4), then [4,5). Choosing [0,3) first would leave less room.
Use BeginnerGreedy.maxNonOverlapping
Cost O(n log n) for sorting.
Check Enumerate all subsets for four intervals and confirm no larger compatible set exists.

Count rooms in a grid

Task Treat # as open in the three rows #.#, #.., and .##. Count four-neighbor connected regions.
Result 3 regions
Reasoning Start a BFS at each unseen open cell. The top-left pair is one region, the top-right cell another, and the bottom-right pair a third.
Use BeginnerGraphs.gridComponents
Cost O(rows * columns), because each cell is visited at most once.
Check Replace the center wall with an open cell and recount the connected regions.

Find the minimum number of coins

Task With unlimited coins of values 1, 3, and 4, form amount 6 using as few coins as possible.
Result 2 coins
Reasoning Let dp[x] be the best answer for amount x. For each coin c <= x, try dp[x-c] + 1. The best choice is 3 + 3.
Use BeginnerDynamicProgramming.minimumCoins
Cost O(amount * number of coin types).
Check Try amount 0, an unreachable amount without coin 1, and a greedy counterexample such as amount 6.

1 Contest setup

Start with a compilable single-file submission and fast input.

1.1 Standalone Main template

Level Beginner
Use when Start a Java judge submission.
Cost Linear in the input size.
Watch for Use the exporter to include required helper classes.

Core idea. Start from a working input/output skeleton so effort stays on the algorithm.
Worked example. Read two integers, add them, and print one line.
Practice. Compile the template before inserting helper classes.

```
import java.io.*;
import java.util.*;

/** Standalone template for judges that require a single Main.java file. */
public class Main {
    static final class FastScanner {
        private final InputStream in = new BufferedInputStream(System.in);
        private final byte[] buffer = new byte[1 << 16];
        private int pos, len;

        private int read() throws IOException {
            if (pos >= len) {
                len = in.read(buffer);
                pos = 0;
                if (len < 0) return -1;
            }
            return buffer[pos++] & 0xff;
        }

        String next() throws IOException {
            int c;
            do { c = read(); } while (c >= 0 && c <= ' ');
            if (c < 0) return null;
            StringBuilder token = new StringBuilder();
            while (c > ' ') { token.append((char) c); c = read(); }
            return token.toString();
        }

        int nextInt() throws IOException { return Integer.parseInt(next()); }
        long nextLong() throws IOException { return Long.parseLong(next()); }
    }

    public static void main(String[] args) throws Exception {
        FastScanner in = new FastScanner();
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
        // Solve the problem here. Read with in.nextInt(), write with out.println().
        out.flush();
    }
}
```

1.2 Reusable fast input

Level Beginner
Use when Read large ASCII-token input.
Cost Linear time; fixed-size buffer.
Watch for Not a Unicode decoder.

Core idea. Buffered token input reduces per-character overhead on large files.
Worked example. Read n, then process exactly n integers with nextInt().
Practice. Test EOF, negative values, and the final token without a newline.

```
package notebook;

import java.io.BufferedInputStream;
import java.io.IOException;
import java.io.InputStream;

/** Fast byte input for ASCII contest tokens. Use PrintWriter/StringBuilder for output. */
public final class ContestIO {
    private final InputStream in;
    private final byte[] buffer = new byte[1 << 16];
    private int pos, len;

    public ContestIO(InputStream input) { in = new BufferedInputStream(input); }

    private int read() throws IOException {
        if (pos >= len) {
            len = in.read(buffer);
            pos = 0;
            if (len < 0) return -1;
        }
        return buffer[pos++] & 0xff;
    }

    /** Returns null at EOF. Tokens are decoded as ASCII. */
    public String next() throws IOException {
        int c;
        do { c = read(); } while (c >= 0 && c <= ' ');
        if (c < 0) return null;
        StringBuilder token = new StringBuilder();
        while (c > ' ') { token.append((char) c); c = read(); }
        return token.toString();
    }

    public int nextInt() throws IOException { return Integer.parseInt(next()); }
    public long nextLong() throws IOException { return Long.parseLong(next()); }
}
```

2 Fundamentals

Arrays, windows, sorting, and binary search come first.

2.1 Loops, arrays, prefix sums, and brute force

Level Beginner
Use when Process a sequence or test all small pairs.
Cost O(n) scans; O(n^2) pair search.
Watch for A prefix array has length n+1 and answers half-open ranges.

Core idea. A direct scan is often the clearest baseline. Prefix sums store the total before each index so one subtraction answers a range.
Worked example. For [3, -2, 5], prefix = [0, 3, 1, 6] and sum [1, 3) = 6 - 3 = 3.
Practice. Compare every range sum against a loop on small random arrays.

```
package notebook;

import java.util.Arrays;

/** Small array patterns that are useful before learning advanced data structures. */
public final class BeginnerArrays {
    private BeginnerArrays() {}

    /** Sum all values. O(n) time, O(1) extra space; throws if the sum exceeds long. */
    public static long sum(int[] values) {
        long result = 0;
        for (int value : values) result = Math.addExact(result, value);
        return result;
    }

    /** Return the index of the first target, or -1. O(n) time, O(1) space. */
    public static int linearSearch(int[] values, int target) {
        for (int i = 0; i < values.length; i++) if (values[i] == target) return i;
        return -1;
    }

    /** Count occurrences of a target. O(n) time, O(1) space. */
    public static int count(int[] values, int target) {
        int result = 0;
        for (int value : values) if (value == target) result++;
        return result;
    }

    /** Reverse an array in place. O(n) time, O(1) extra space. */
    public static void reverse(int[] values) {
        for (int left = 0, right = values.length - 1; left < right; left++, right--) {
            int temp = values[left]; values[left] = values[right]; values[right] = temp;
        }
    }

    /** Inclusive prefix sums with prefix[0] = 0. O(n) time and space. */
    public static long[] prefixSums(int[] values) {
        long[] prefix = new long[values.length + 1];
        for (int i = 0; i < values.length; i++) prefix[i + 1] = Math.addExact(prefix[i], values[i]);
        return prefix;
    }

    /** Sum [left, right) using a prefix array. O(1) time. */
    public static long rangeSum(long[] prefix, int left, int right) {
        if (left < 0 || right < left || right >= prefix.length) throw new IndexOutOfBoundsException();
        return prefix[right] - prefix[left];
    }

    /** Two-sum by trying every pair; returns indices or an empty array. O(n^2) time. */
    public static int[] pairWithSum(int[] values, long target) {
        for (int i = 0; i < values.length; i++)
            for (int j = i + 1; j < values.length; j++)
                if ((long) values[i] + values[j] == target) return new int[] {i, j};
        return new int[0];
    }

    /** Copy and sort values without modifying the input. O(n log n) time. */
```

```
    public static int[] sortedCopy(int[] values) {
        int[] result = values.clone();
        Arrays.sort(result);
        return result;
    }
}
```

2.2 Maps, sets, and stacks

Level Beginner
Use when Count frequencies, remove duplicates, or check nested delimiters.
Cost Expected O(n) time for hash collections; O(n) stack space.
Watch for HashMap and HashSet have no guaranteed iteration order.

Core idea. A map stores a value for each key, a set stores membership, and a stack reverses the order in which items are processed.
Worked example. For [4,2,4], the frequency map stores 4 to 2 and 2 to 1. The string ([{}]) is balanced.
Practice. Solve a frequency-count problem with both a map and a sorted array, then compare the tradeoffs.

```
package notebook;

import java.util.ArrayDeque;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/** Everyday Java collections: map, set, and stack. */
public final class BeginnerCollections {
    private BeginnerCollections() {}

    /** Count each integer with a hash map. Expected O(n) time and O(distinct) space. */
    public static Map<Integer, Integer> frequencies(int[] values) {
        Map<Integer, Integer> result = new HashMap<>();
        for (int value : values) result.merge(value, 1, Integer::sum);
        return result;
    }

    /** Count distinct integers with a hash set. Expected O(n) time and space. */
    public static int distinctCount(int[] values) {
        Set<Integer> seen = new HashSet<>();
        for (int value : values) seen.add(value);
        return seen.size();
    }

    /** Check balanced (), [], and {} brackets. O(n) time, O(n) stack space. */
    public static boolean balancedBrackets(String text) {
        ArrayDeque<Character> stack = new ArrayDeque<>();
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (ch == '(' || ch == '[' || ch == '{') stack.push(ch);
            else if (ch == ')' || ch == ']' || ch == '}') {
                if (stack.isEmpty()) return false;
                char open = stack.pop();
                if (ch == ')' && open != '(' || ch == ']' && open != '[' || ch == '}' && open != '{')
                    return false;
            } else throw new IllegalArgumentException("expected brackets only");
        }
        return stack.isEmpty();
    }
}
```

2.3 Recursion and exhaustive subsets

Level Beginner
Use when Enumerate all choices when n is small.
Cost O(n) factorial; O(2^n) subset search.
Watch for Exponential search is only practical for small n; recursion uses stack space.

Core idea. A recursive function solves a smaller instance, reaches a base case, then combines answers. For each subset element, branch into skip and take.
Worked example. For [1,2,3] and target 3, subsets [3] and [1,2] both match.
Practice. Draw the recursion tree for three values and mark its base cases.

```
package notebook;

/** Recursion and exhaustive search when the input is small enough. */
public final class BeginnerBacktracking {
    private BeginnerBacktracking() {}

    /** Factorial using a smaller subproblem. O(n) time and stack; n must be in [0,20]. */
    public static long factorial(int n) {
        if (n < 0 || n > 20) throw new IllegalArgumentException("n must be in [0,20]");
        if (n <= 1) return 1;
        return n * factorial(n - 1);
    }

    /** Count subsets whose sum equals target. O(2^n) time, O(n) stack; n <= 25. */
    public static long countSubsetsWithSum(int[] values, long target) {
        if (values.length > 25) throw new IllegalArgumentException("too many subsets");
        return count(values, 0, 0, target);
    }

    private static long count(int[] values, int index, long sum, long target) {
        if (index == values.length) return sum == target ? 1 : 0;
        return count(values, index + 1, sum, target)
                + count(values, index + 1, Math.addExact(sum, values[index]), target);
    }
}
```

2.4 Binary search and bounds

Level Beginner
Use when Search sorted data or a monotone answer predicate.
Cost O(log n) search.
Watch for Sorted input and monotonicity are preconditions.

Core idea. Binary search keeps an interval that is guaranteed to contain the answer.
Worked example. For sorted [1, 2, 2, 5], lowerBound(2) is 1 and upperBound(2) is 3.
Practice. Test an empty array and targets below and above every element.

```
package notebook;

import java.util.function.LongPredicate;

/** Search and sorting routines. Input to searches must be sorted ascending. */
public final class SearchingSorting {
    private SearchingSorting() {}

    /** O(log n); returns any matching index, or -1. */
    public static int binarySearch(int[] a, int key) {
        int lo = 0, hi = a.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] == key) return mid;
            if (a[mid] < key) lo = mid + 1;
            else hi = mid - 1;
```

```
        }
        return -1;
    }

    /** O(log n); returns any matching index, or -1. */
    public static int ternarySearch(int[] a, int key) {
        int lo = 0, hi = a.length - 1;
        while (lo <= hi) {
            int m1 = lo + (hi - lo) / 3;
            int m2 = hi - (hi - lo) / 3;
            if (a[m1] == key) return m1;
            if (a[m2] == key) return m2;
            if (key < a[m1]) hi = m1 - 1;
            else if (key > a[m2]) lo = m2 + 1;
            else { lo = m1 + 1; hi = m2 - 1; }
        }
        return -1;
    }

    /** O(n + 65536) counting sort for Java UTF-16 char units. */
    public static void countingSort(char[] a) {
        int[] count = new int[Character.MAX_VALUE + 1];
        for (char c : a) count[c]++;
        int i = 0;
        for (int c = 0; c < count.length; c++) {
            while (count[c]-- > 0) a[i++] = (char) c;
        }
    }

    /** Lower bound: first index i with a[i] >= key. O(log n). */
    public static int lowerBound(int[] a, int key) {
        int lo = 0, hi = a.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] < key) lo = mid + 1;
            else hi = mid;
        }
        return lo;
    }

    /** First index with a[i] > key in a sorted array. O(log n). */
    public static int upperBound(int[] a, int key) {
        int lo = 0, hi = a.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] <= key) lo = mid + 1;
            else hi = mid;
        }
        return lo;
    }

    /** First x in nonnegative [low,high] satisfying a monotone false-then-true predicate. */
    public static long firstTrue(long low, long high, LongPredicate predicate) {
        if (low < 0 || low > high || !predicate.test(high)) throw new IllegalArgumentException("no true value in
     range");
        while (low < high) {
            long mid = low + ((high - low) >>> 1);
            if (predicate.test(mid)) high = mid;
            else low = mid + 1;
        }
        return low;
    }
}
```

2.5 Interval scheduling and covering

Level Beginner
Use when Select compatible intervals or cover intervals with few points.
Cost O(n log n) from sorting.
Watch for Scheduling uses half-open intervals; point covering uses closed intervals.

Core idea. Sort by earliest finishing time. Choosing an early finish leaves as much room as possible for later intervals.
Worked example. Among [0,3), [1,2), [2,4), choose the last two.
Practice. Try to prove the choice by exchanging an optimal first interval with the greedy one.

```
package notebook;

import java.util.Arrays;
import java.util.Comparator;

/** Greedy choices with simple exchange-argument proofs. */
public final class BeginnerGreedy {
    private BeginnerGreedy() {}

    /** Maximum number of non-overlapping [start,end) intervals. O(n log n) time. */
    public static int maxNonOverlapping(long[][] intervals) {
        long[][] sorted = new long[intervals.length][];
        for (int i = 0; i < intervals.length; i++) {
            if (intervals[i].length != 2 || intervals[i][0] > intervals[i][1])
                throw new IllegalArgumentException("invalid interval");
            sorted[i] = intervals[i].clone();
        }
        Arrays.sort(sorted, Comparator.comparingLong((long[] interval) -> interval[1])
                .thenComparingLong(interval -> interval[0]));
        int chosen = 0;
        long lastEnd = Long.MIN_VALUE;
        for (long[] interval : sorted) if (interval[0] >= lastEnd) {
            chosen++;
            lastEnd = interval[1];
        }
        return chosen;
    }

    /** Minimum points that cover closed intervals [left,right]. O(n log n) time. */
    public static int pointsToCover(long[][] intervals) {
        long[][] sorted = new long[intervals.length][];
        for (int i = 0; i < intervals.length; i++) {
            if (intervals[i].length != 2 || intervals[i][0] > intervals[i][1])
                throw new IllegalArgumentException("invalid interval");
            sorted[i] = intervals[i].clone();
        }
        Arrays.sort(sorted, Comparator.comparingLong(interval -> interval[1]));
        int count = 0;
        long lastPoint = 0;
        boolean hasPoint = false;
        for (long[] interval : sorted) if (!hasPoint || interval[0] > lastPoint) {
            lastPoint = interval[1];
            hasPoint = true;
            count++;
        }
        return count;
    }
}
```

3 Dynamic programming

State, transition, base case, and iteration order should be explicit.

3.1 First dynamic programs

Level Beginner
Use when Reuse answers to smaller states for sequences, coin change, or grids.
Cost O(n), O(amount * coin types), or O(rows * columns).
Watch for Path counts can overflow long; coin denominations must be positive.

Core idea. Define the state, base case, transition, and answer before writing loops. An iterative table avoids repeated recursion.
Worked example. For coins 1, 3, 4, amount 6 needs two coins: 3 + 3.
Practice. Write a table by hand for a small input and compare it with the program.

```
package notebook;

import java.util.Arrays;

/** First DP examples: identify a state, base case, transition, and answer. */
public final class BeginnerDynamicProgramming {
    private BeginnerDynamicProgramming() {}

    /** Fibonacci F(n), with F(0)=0 and F(1)=1. O(n) time; n must be at most 92. */
    public static long fibonacci(int n) {
        if (n < 0 || n > 92) throw new IllegalArgumentException("n must be in [0,92]");
        long previous = 0, current = 1;
        for (int i = 0; i < n; i++) { long next = previous + current; previous = current; current = next; }
        return previous;
    }

    /** Minimum coins for an amount, or -1 if impossible. O(amount * coins.length) time. */
    public static int minimumCoins(int[] coins, int amount) {
        if (amount < 0) throw new IllegalArgumentException("negative amount");
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;
        for (int value = 1; value <= amount; value++)
            for (int coin : coins) {
                if (coin <= 0) throw new IllegalArgumentException("coins must be positive");
                if (coin <= value) dp[value] = Math.min(dp[value], dp[value - coin] + 1);
            }
        return dp[amount] > amount ? -1 : dp[amount];
    }

    /** Number of paths in a grid, moving right or down around blocked cells. O(rows*columns). */
    public static long gridPaths(boolean[][] blocked) {
        int rows = blocked.length;
        if (rows == 0) return 0;
        int columns = blocked[0].length;
        for (boolean[] row : blocked) if (row.length != columns) throw new IllegalArgumentException("ragged grid");
        if (columns == 0) return 0;
        long[] dp = new long[columns];
        dp[0] = blocked[0][0] ? 0 : 1;
        for (int row = 0; row < rows; row++)
            for (int column = 0; column < columns; column++) {
                if (blocked[row][column]) dp[column] = 0;
                else if (column > 0) dp[column] = Math.addExact(dp[column], dp[column - 1]);
            }
        return dp[columns - 1];
    }
}
```

4 Graphs

Distinguish directed and undirected edges before choosing an algorithm.

4.1 Adjacency lists, BFS, and grid flood fill

Level Beginner
Use when Explore an unweighted graph or count regions in a grid.
Cost O(V+E) for graphs; O(rows * columns) for grids.
Watch for For undirected graphs, insert both directions; BFS distance counts edges.

Core idea. An adjacency list stores each vertex's neighbors. BFS visits vertices in layers, so its first visit gives the shortest unweighted distance.
Worked example. For edges 0-1 and 1-2, BFS from 0 returns distances 0, 1, 2.
Practice. Draw a graph with an unreachable vertex and check the returned -1 distance.

```
package notebook;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/** First graph routines using adjacency lists and an explicit queue. */
public final class BeginnerGraphs {
    private BeginnerGraphs() {}

    /** Build an undirected adjacency list from zero-indexed edges. O(V+E) time and space. */
    public static List<List<Integer>> undirectedGraph(int vertices, int[][] edges) {
        if (vertices < 0) throw new IllegalArgumentException("negative vertex count");
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < vertices; i++) graph.add(new ArrayList<>());
        for (int[] edge : edges) {
            if (edge.length != 2) throw new IllegalArgumentException("edge must contain two vertices");
            graph.get(edge[0]).add(edge[1]);
            graph.get(edge[1]).add(edge[0]);
        }
        return graph;
    }

    /** BFS distances from source; -1 means unreachable. O(V+E) time. */
    public static int[] distances(List<List<Integer>> graph, int source) {
        int[] distance = new int[graph.size()];
        Arrays.fill(distance, -1);
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        distance[source] = 0;
        queue.add(source);
        while (!queue.isEmpty()) {
            int vertex = queue.remove();
            for (int next : graph.get(vertex)) if (distance[next] == -1) {
                distance[next] = distance[vertex] + 1;
                queue.add(next);
            }
        }
        return distance;
    }

    /** Count four-neighbor components of true cells. O(rows*columns) time and space. */
    public static int gridComponents(boolean[][] open) {
        int rows = open.length;
        if (rows == 0) return 0;
        int columns = open[0].length;
        for (boolean[] row : open) if (row.length != columns) throw new IllegalArgumentException("ragged grid");
        boolean[][] seen = new boolean[rows][columns];
        int[] dr = {-1, 1, 0, 0}, dc = {0, 0, -1, 1};
        ArrayDeque<int[]> queue = new ArrayDeque<>();
```

```
        int components = 0;
        for (int row = 0; row < rows; row++) for (int column = 0; column < columns; column++) {
            if (!open[row][column] || seen[row][column]) continue;
            components++;
            seen[row][column] = true;
            queue.add(new int[] {row, column});
            while (!queue.isEmpty()) {
                int[] cell = queue.remove();
                for (int direction = 0; direction < 4; direction++) {
                    int nextRow = cell[0] + dr[direction], nextColumn = cell[1] + dc[direction];
                    if (nextRow < 0 || nextRow >= rows || nextColumn < 0 || nextColumn >= columns
                            || !open[nextRow][nextColumn] || seen[nextRow][nextColumn]) continue;
                    seen[nextRow][nextColumn] = true;
                    queue.add(new int[] {nextRow, nextColumn});
                }
            }
        }
        return components;
    }
}
```

5 Strings

Choose a one-pattern, many-pattern, palindrome, or suffix method.

5.1 Characters, frequencies, and direct matching

Level Beginner
Use when Count letters, test palindromes, or match a short pattern.
Cost O(n) scans; O(nm) direct matching.
Watch for Java char values are UTF-16 code units; the frequency helper accepts a-z only.

Core idea. A string is an indexed sequence. Start with direct comparisons before choosing a faster string algorithm.
Worked example. The pattern aa occurs three times in aaaa because matches can overlap.
Practice. Test an empty text, one character, and overlapping matches.

```
package notebook;

/** Character and substring patterns for introductory contest problems. */
public final class BeginnerStrings {
    private BeginnerStrings() {}

    /** Compare characters from both ends. O(n) time, O(1) extra space. */
    public static boolean isPalindrome(String text) {
        for (int left = 0, right = text.length() - 1; left < right; left++, right--)
            if (text.charAt(left) != text.charAt(right)) return false;
        return true;
    }

    /** Count lowercase English letters. O(n) time, O(1) space; rejects other characters. */
    public static int[] lowercaseFrequencies(String text) {
        int[] frequency = new int[26];
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (ch < 'a' || ch > 'z') throw new IllegalArgumentException("expected a-z");
            frequency[ch - 'a']++;
        }
        return frequency;
    }

    /** Count exact occurrences, including overlaps. O(n*m) time. */
    public static int countOccurrences(String text, String pattern) {
        if (pattern.isEmpty()) throw new IllegalArgumentException("pattern must not be empty");
```

```
        int count = 0;
        for (int start = 0; start + pattern.length() <= text.length(); start++) {
            int matched = 0;
            while (matched < pattern.length() && text.charAt(start + matched) == pattern.charAt(matched))
                matched++;
            if (matched == pattern.length()) count++;
        }
        return count;
    }

    /** Longest shared prefix of two strings. O(min(n,m)) time. */
    public static String commonPrefix(String first, String second) {
        int length = 0;
        while (length < first.length() && length < second.length()
                && first.charAt(length) == second.charAt(length)) length++;
        return first.substring(0, length);
    }
}
```

6 Number theory and combinatorics

Use exact or modular arithmetic according to the requested result.

6.1 GCD, trial division, divisors, and powers

Level Beginner
Use when Solve small divisibility and integer arithmetic tasks.
Cost GCD O(log n); trial division O(sqrt n); power O(log exponent).
Watch for Exact power reports long overflow; inputs to gcd must be nonnegative.

Core idea. A divisor larger than the square root has a matching smaller divisor. Euclid repeatedly replaces (a,b) by (b,a mod b).
Worked example. The divisors of 36 come in pairs around 6, giving nine positive divisors.
Practice. Compare trial-division results with direct enumeration for small values.

```
package notebook;

/** Elementary integer math before modular arithmetic and number theory. */
public final class BeginnerMath {
    private BeginnerMath() {}

    /** Euclid's algorithm for nonnegative inputs. O(log min(a,b)) time. */
    public static long gcd(long a, long b) {
        if (a < 0 || b < 0) throw new IllegalArgumentException("expected nonnegative values");
        while (b != 0) { long next = a % b; a = b; b = next; }
        return a;
    }

    /** Test primality by trial division. O(sqrt(n)) time, O(1) space. */
    public static boolean isPrime(long value) {
        if (value < 2) return false;
        for (long divisor = 2; divisor <= value / divisor; divisor++)
            if (value % divisor == 0) return false;
        return true;
    }

    /** Count positive divisors of a positive integer. O(sqrt(n)) time. */
    public static long divisorCount(long value) {
        if (value <= 0) throw new IllegalArgumentException("expected a positive value");
        long count = 0;
        for (long divisor = 1; divisor <= value / divisor; divisor++)
            if (value % divisor == 0) count += divisor == value / divisor ? 1 : 2;
        return count;
```

```
    }

    /** Compute base^exponent for a nonnegative exponent. O(log exponent) time. */
    public static long power(long base, long exponent) {
        if (exponent < 0) throw new IllegalArgumentException("negative exponent");
        long result = 1;
        while (exponent != 0) {
            if ((exponent & 1) != 0) result = Math.multiplyExact(result, base);
            exponent >>= 1;
            if (exponent != 0) base = Math.multiplyExact(base, base);
        }
        return result;
    }
}
```